

C++ Programming Workbook

1. Functions & Overloading

Basic building blocks: modularity and reusing names for different parameter types[cite: 1].

EX 1.1

Write a function `calculateOctave` that takes a center frequency (double) and returns the upper band limit (frequency * 1.414).

EX 1.2

Overload `logSignal`. One version takes a `double` (representing a single PSD value) and prints it. The second version takes a `double` and an `int` (representing a band index) and prints "Band [index]: [value]".

2. Templates & Specialization

Generic programming: writing code that works for any data type[cite: 1].

EX 2.1

Create a function template `findAbsMax` that takes two arguments of the same type and returns the one with the larger absolute value. (Recall: Overloading - ensure it works for integers and floats).

EX 2.2

Create a template `printData<T>`. Provide a **Template Specialization** for `const char*` that wraps the text in square brackets, while the general template just prints the data normally.

3. Lambda Functions

Inline anonymous functions that can capture local variables[cite: 1].

EX 3.1

Define a lambda that captures a `threshold` variable by value. The lambda should take a `double value` and return `true` if `value > threshold`.

EX 3.2

Create a function that takes a `double signal` and a lambda as a "processor". Use the lambda to subtract a `max_val` from the signal (simulating the subtraction of the maximum PSD value).

4. The 'Requires' Clause

Constraints for templates to ensure they only accept valid types[cite: 1].

EX 4.1

Write a template function `addSignals` that uses a `requires` clause to ensure both parameters are arithmetic types (numbers). Use `std::is_arithmetic_v<T>`.

EX 4.2

Combine **Specialization** and **Requires**: Create a function that prints a signal. Use `requires` to only allow floating-point types. If a user tries to pass a string, the compiler should fail cleanly.

5. Function Pointers & Arguments

Treating functions like data by passing their addresses to other functions[cite: 1].

EX 5.1

Write two functions: `applyGain` and `applyAttenuation`. Write a third function `processBuffer` that takes a `double` value and a **function pointer** to choose which process to apply.

EX 5.2

DSP Mix: Write a function that accepts a function pointer. Use this pointer to pass a **Lambda** that performs a subtraction of a max PSD value. Ensure the main function also uses a `requires` clause for type safety[cite: 1, 2].

6. Fold Expressions & Arg Packs

Handling a variable number of arguments (variadic templates) using concise syntax[cite: 1].

EX 6.1

Write a variadic template `averageSignal` that takes any number of doubles using an **argument pack**. Use a **fold expression** to sum them all up and divide by the count (use `sizeof...(args)`).

EX 6.2

Create a function `allAboveThreshold` that takes a threshold and a variadic pack of values. Use a fold expression with the `&&` operator to return true ONLY if every value in the pack is above the threshold.

7. Constexpr

Enforcing compile-time evaluation for maximum efficiency[cite: 1].

EX 7.1

Create a `constexpr` function that calculates the center frequency of the Nth octave band (Formula: $1000 * 2^N$). Verify it works at compile time by using it to size a raw array: `int data[(int)getFreq(2)];` .

EX 7.2

The Ultimate Mix: Create a `constexpr` function that calculates a coefficient. Pass this coefficient into a **Variadic Template** function that uses a **Fold Expression** to apply the coefficient to a list of signal values provided via a **Lambda**[cite: 1, 2].